Diagrammes de séquences

Par Dr Lamia BEN GHEZAIEL

ISAMM 2015-2016

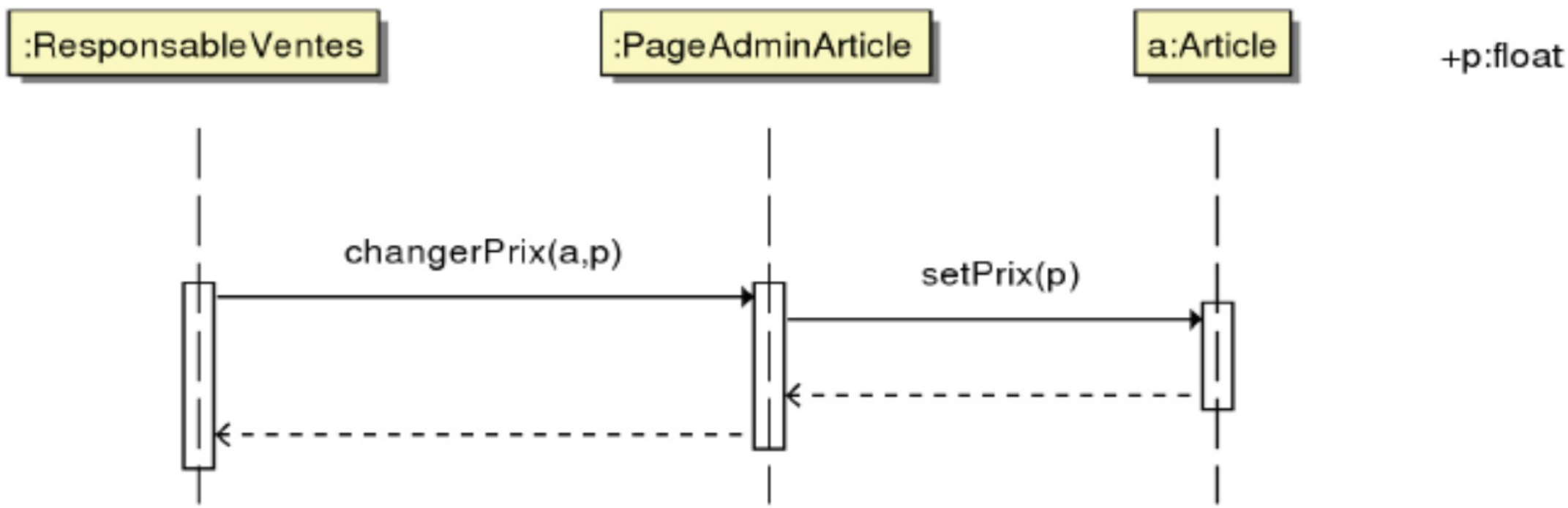
Objectifs

- Les **diagrammes de cas** d'utilisation modélisent à **QUOI** sert le système, en organisant les interactions possibles avec les acteurs.
- Les **diagrammes de classes** permettent de spécifier la structure et les liens entre les objets dont le système est composé : ils spécifie **QUI** sera à l'œuvre dans le système pour réaliser les fonctionnalités décrites par les diagrammes de cas d'utilisation.
- Les **diagrammes de séquences** permettent de décrire **COMMENT** les éléments du système interagissent entre eux et avec les acteurs.
  - Les objets au cœur d'un système interagissent en s'échangent des messages.
  - Les acteurs interagissent avec le système au moyen d'IHM (Interfaces Homme-Machine).

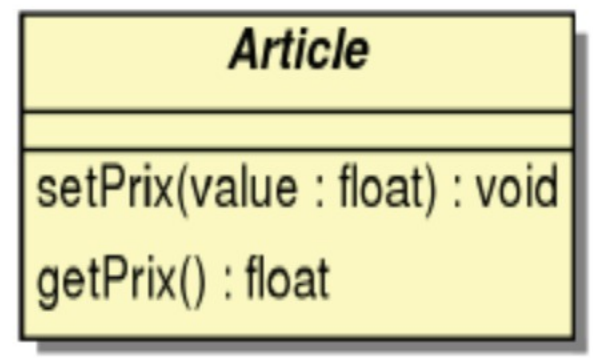
Exemple d'interaction

- Cas d'utilisation

- Diagramme de séquences correspondant :

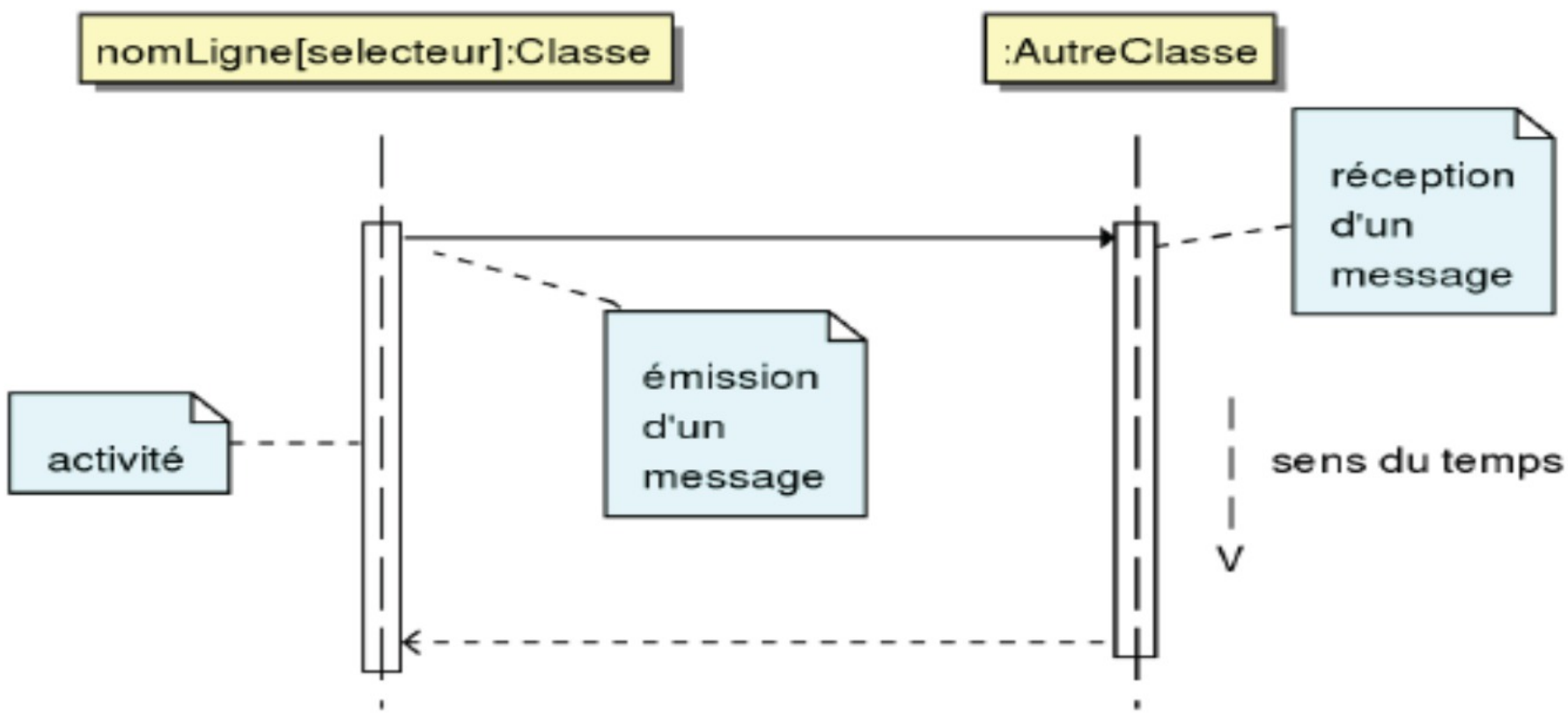


- Opérations nécessaires dans le diagrammes de classes



Ligne de vie

- Une ligne de vie représente un participant à une interaction (objet ou acteur).
- nomLigneDeVie [selecteur]: nomClasseOuActeur
- Dans le cas d'une collection de participants, un sélecteur permet de choisir un objet parmi n (par exemple objets[2]).



Les messages

- Les principales informations contenues dans un diagramme de séquences sont les messages échangés entre les lignes de vie, présentés dans un ordre chronologique.
  - Un message définit une communication particulière entre des lignes de vie (objets ou acteurs).
  - Plusieurs types de messages existent, dont les plus courants :
    - l'envoi d'un signal ;
    - l'invocation d'une opération (appel de méthode) ;
    - la création ou la destruction d'un objet.
  - La réception des messages provoque une période d'activité (rectangle vertical sur la ligne de vie) marquant le traitement du message (spécification d'exécution dans le cas d'un appel de méthode).

Principaux types de messages

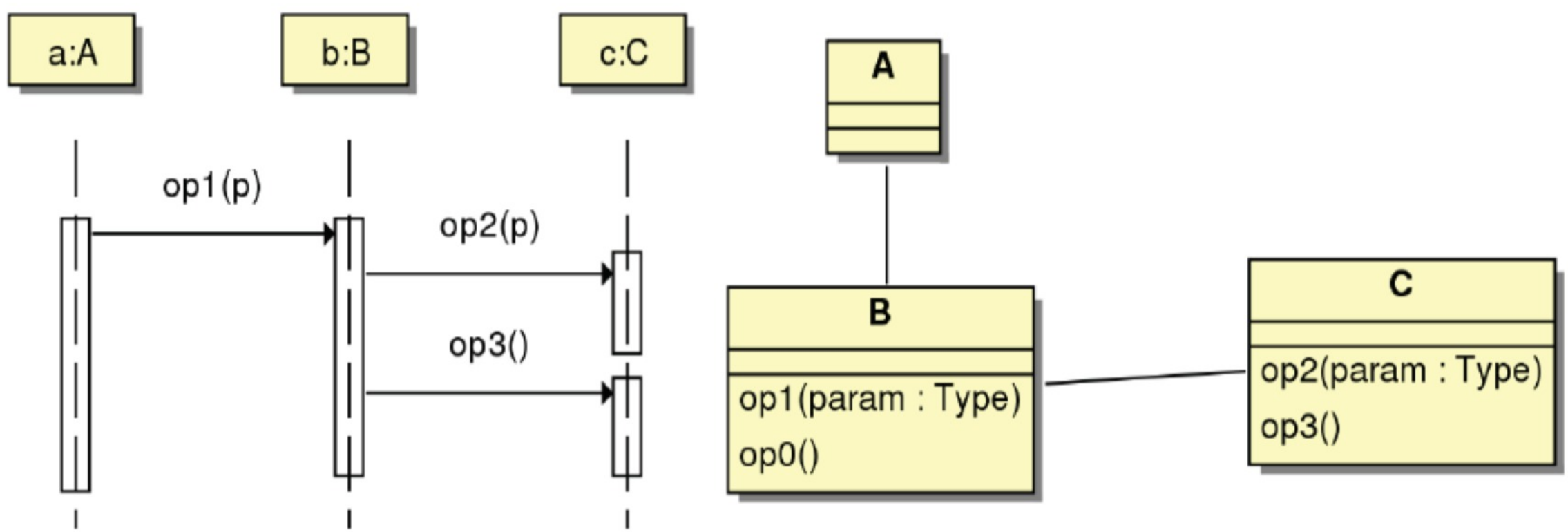
- Un message **synchrone** bloque l'expéditeur jusqu'à la réponse du destinataire. Le flot de contrôle passe de l'émetteur au récepteur.
  - Typiquement : appel de méthode
    - Si un objet A invoque une méthode d'un objet B, A reste bloqué tant que B n'a pas terminé.

    - On peut associer aux messages d'appel de méthode un message de retour (en pointillés) marquant la reprise du contrôle par l'objet émetteur du message synchrone.
- Un message **asynchrone** n'est pas bloquant pour l'expéditeur. Le message envoyé peut être pris en compte par le récepteur à tout moment ou ignoré.
  - Typiquement : envoi de signal (voir stéréotype de classe signal).

Correspondance messages / opérations

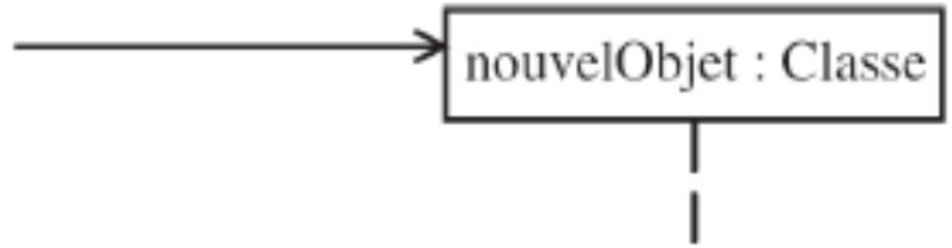
- Les messages synchrones correspondent à des opérations dans le diagramme de classes.

Envoyer un message et attendre la réponse pour poursuivre son activité revient à invoquer une méthode et attendre le retour pour poursuivre ses traitements.

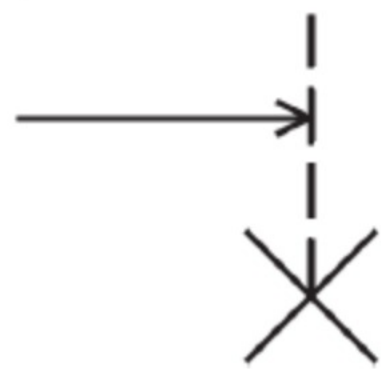


Création et destruction de lignes de vie

- La création d'un objet est matérialisée par une flèche qui pointe sur le sommet d'une ligne de vie.
  - On peut aussi utiliser un message asynchrone ordinaire portant le nom « create »



- La destruction d'un objet est matérialisée par une croix qui marque la fin de la ligne de vie de l'objet.



Messages complets, perdus et trouvés

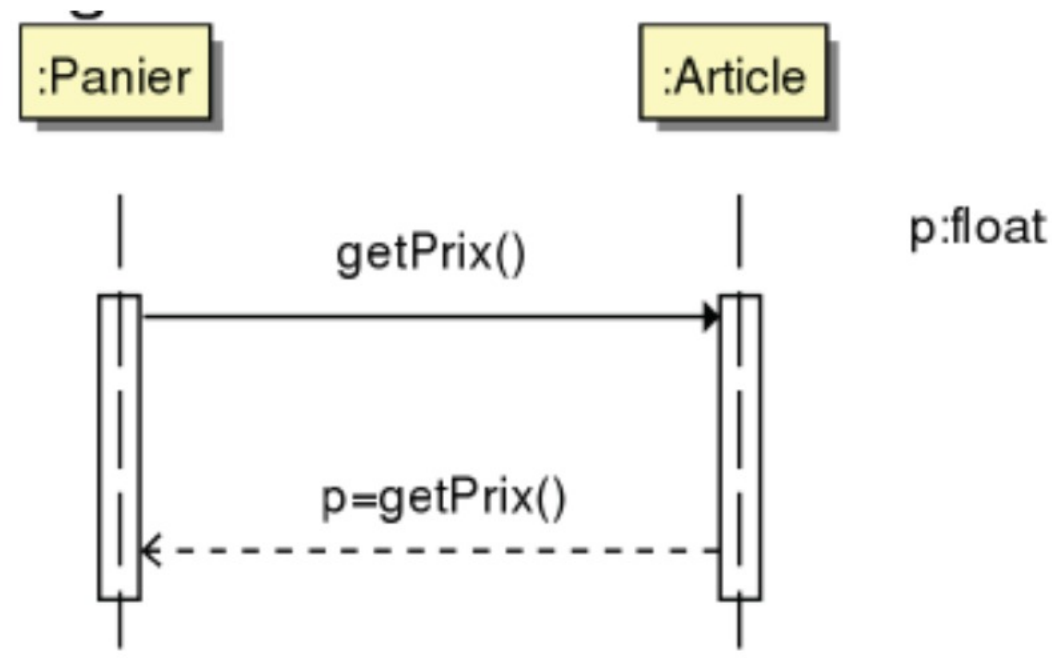
- Un **message complet** est tel que les événements d'envoi et de réception sont connus.
  - Un message complet est représenté par une flèche partant d'une ligne de vie et arrivant à une autre ligne de vie.
- Un **message perdu** est tel que l'événement d'envoi est connu, mais pas l'événement de réception.

  - La flèche part d'une ligne de vie mais arrive sur un cercle indépendant marquant la méconnaissance du destinataire.
  - Exemple : broadcast.
- Un **message trouvé** est tel que l'événement de réception est connu, mais pas l'événement d'émission.

Messages de retour

- Le récepteur d'un message synchrone rend la main à l'émetteur du message en lui envoyant un message de retour

- Les messages de retour sont optionnels : la fin de la période d'activité marque également la fin de l'exécution d'une méthode.
- Ils sont utilisés pour spécifier le résultat de la méthode invoquée.

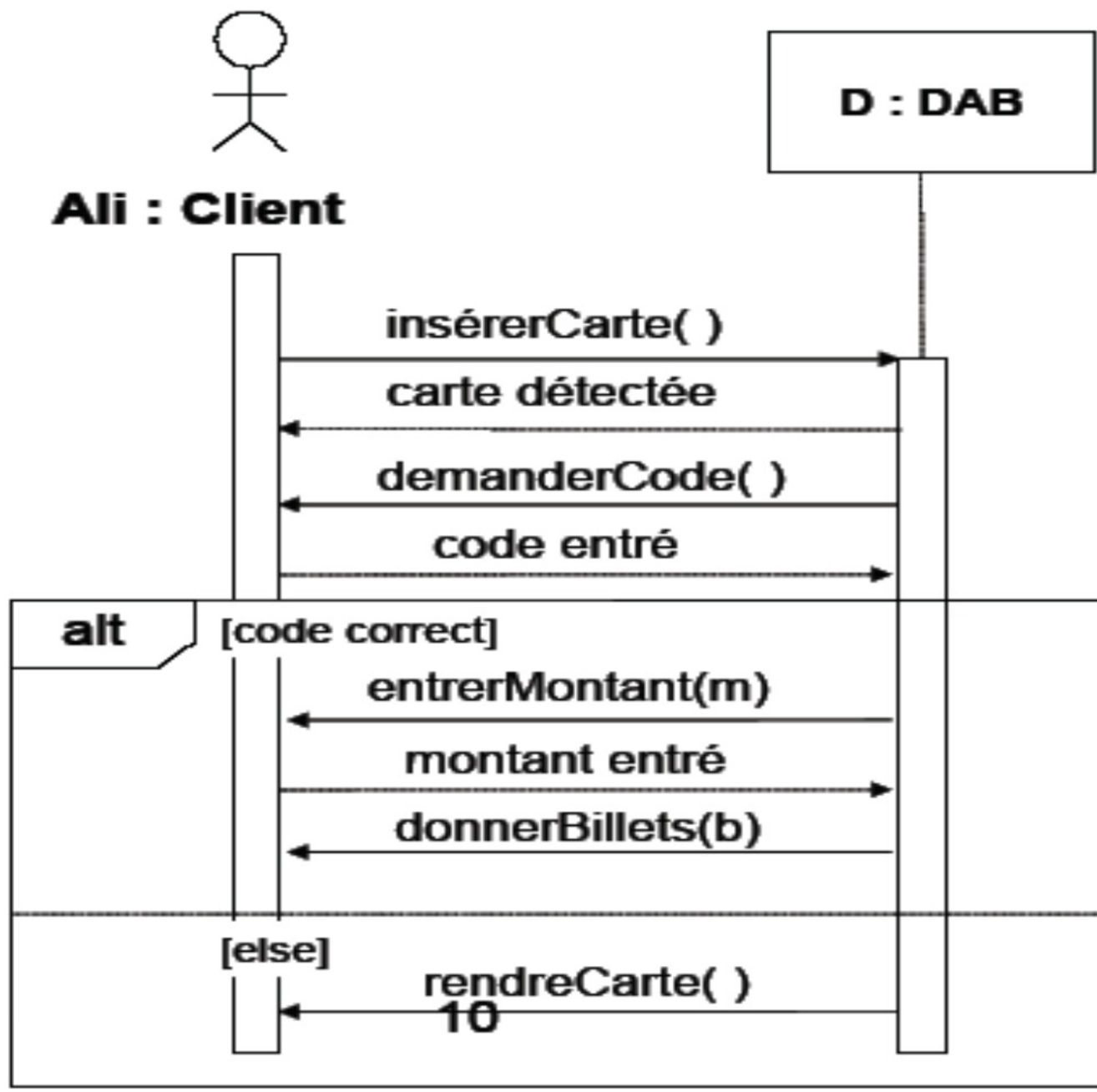


Le retour des messages asynchrones s'effectue par l'envoi de nouveaux messages asynchrones.

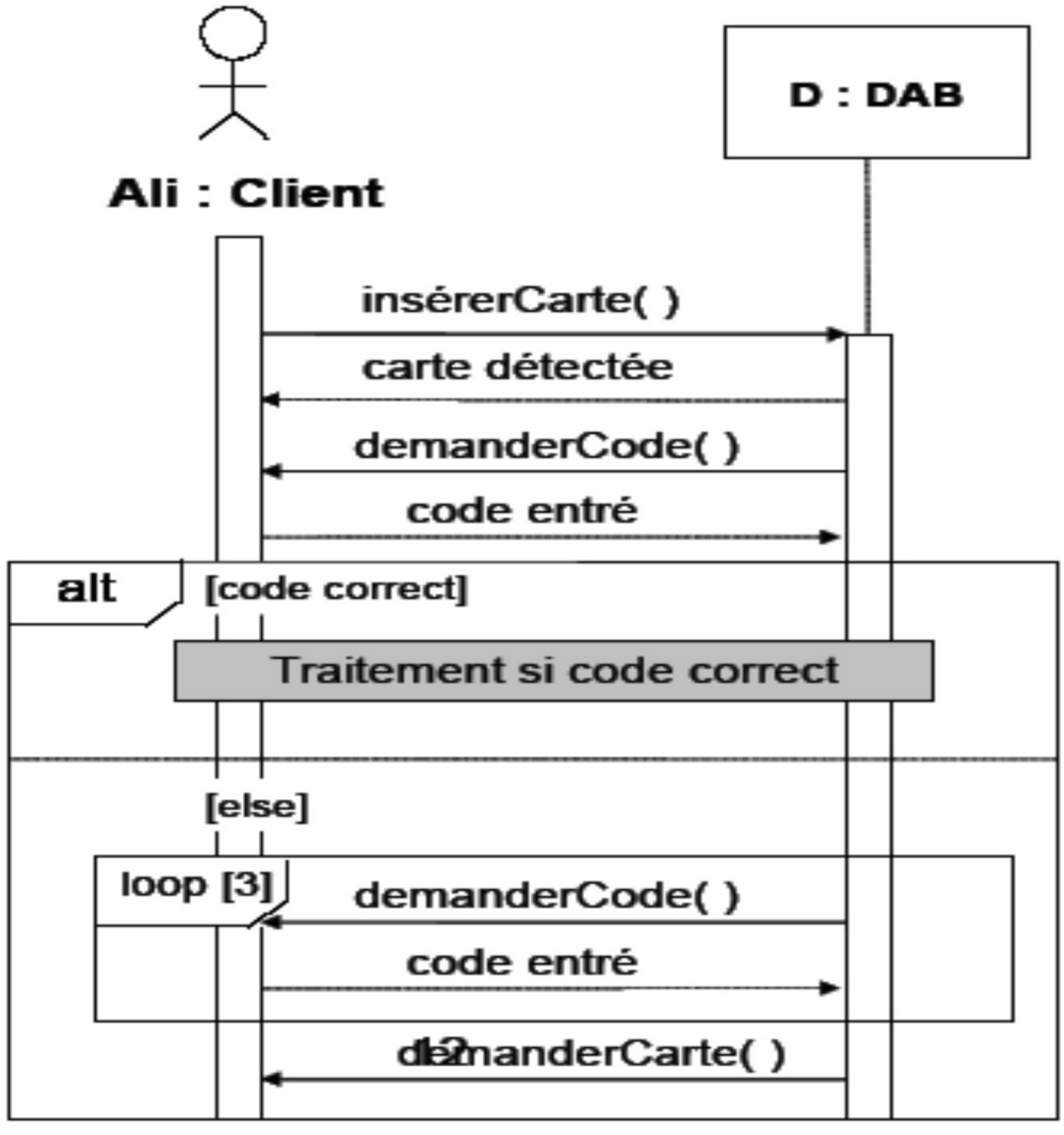
Fragment combiné

- Un fragment combiné permet de décomposer une interaction complexe en fragments suffisamment simples pour être compris.
  - Recombiner les fragments restitue la complexité.
  - Syntaxe complète avec UML 2 : représentation complète de processus avec un langage simple (ex : processus parallèles).
- Un fragment combiné se représente de la même façon qu'une interaction. Il est représenté un rectangle dont le coin supérieur gauche contient un pentagone.
  - Dans le pentagone figure le type de la combinaison (appelé « opérateur d'interaction »).

Exemple de fragment avec l'opérateur conditionnel



Opérateur de boucle

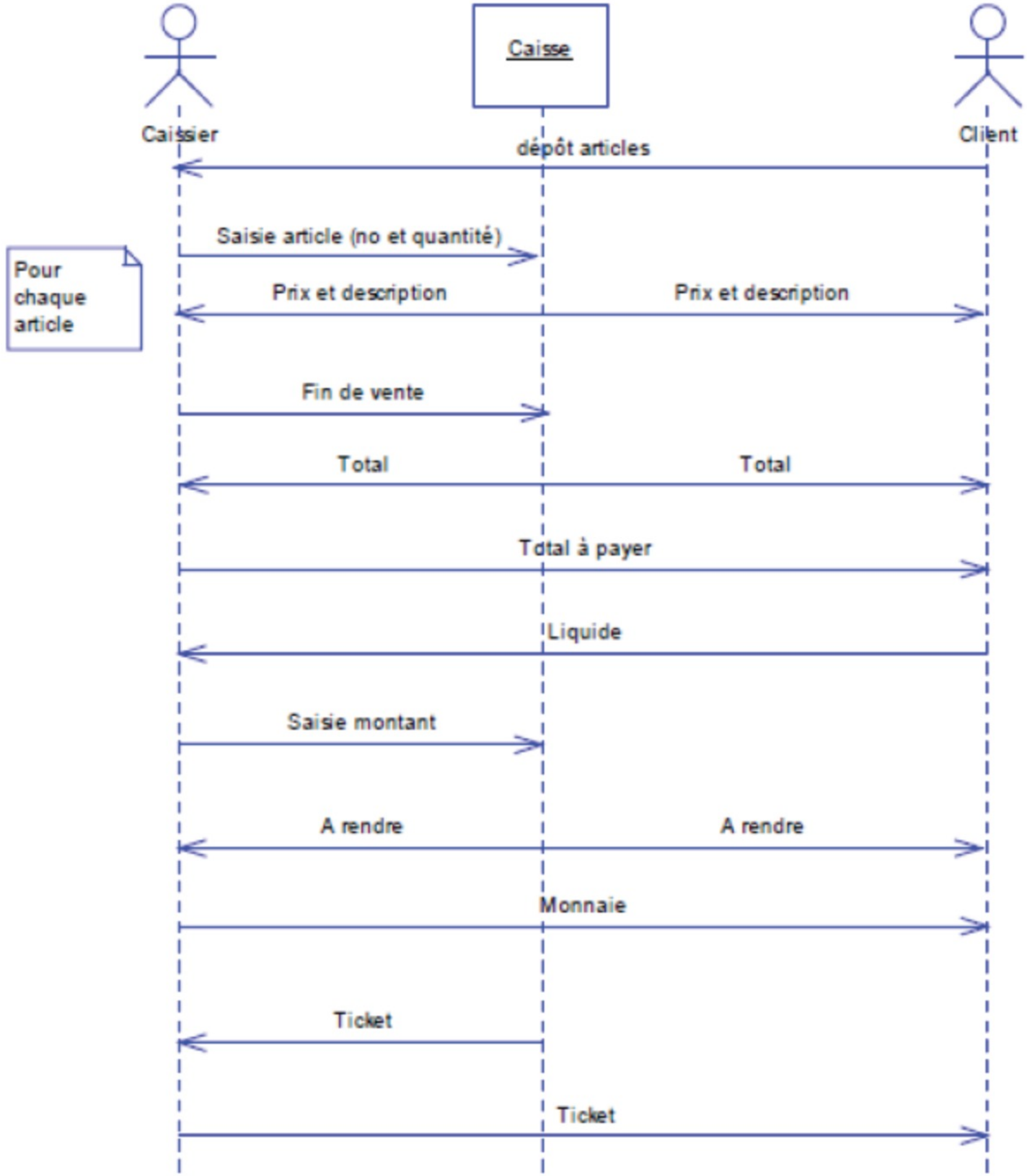


Exercice D'application

Le déroulement normal d'utilisation d'une caisse de supermarché est le suivant :

- un client arrive à la caisse avec ses articles à payer
- le caissier enregistre le numéro d'identification de chaque article, ainsi que la quantité si elle est supérieure à 1
- la caisse affiche le prix de chaque article et son libellé
- lorsque tous les achats sont enregistrés, le caissier signale la fin de la vente
- la caisse affiche le total des achats
- le caissier annonce au client le montant total à payer
- le client choisit son mode de paiement
  - liquide : le caissier encaisse l'argent, la caisse indique le montant à rendre au client
  - chèque : le caissier note le numéro de pièce d'identité du client
  - carte de crédit : la demande d'autorisation est envoyée avant la saisie
- la caisse enregistre la vente et l'imprime
- le caissier donne le ticket de caisse au client

Modéliser cette situation à l'aide d'un diagramme de séquences en ne prenant en compte que le cas du paiement en liquide.


Caisse
Caissier
Client
dépôt articles
Saisie article (no et quantité)
Pour chaque article
Prix et description
Prix et description
Fin de vente
Total
Total
Total à payer
Liquide
Saisie montant
A rendre
A rendre
Monnaie
Ticket
Ticket